

Day-68-Basic API Data Collection Python

1. Day Number

Day 68

2. Topic Name

API Request and JSON Response

3. Connection to Yesterday

Yesterday, on **Day 67**, you learned about **JSON data in Python**.

You learned that JSON can store information using structures such as:

```
{  
  "name": "Ravi",  
  "city": "Bengaluru"  
}
```

Today, you will see an important real-world use of JSON:

Many websites and web APIs send data to Python programs in JSON format.

The basic flow is:

```
Python program  
  ↓  
Send API request  
  ↓  
API server  
  ↓  
JSON response  
  ↓  
Python reads JSON
```

4. Important Topics

Today's important ideas are:

- **API**
- **URL**
- **request**
- **response**
- **status code**
- **JSON response**
- **requests library**

You only need to understand the basic request-response process today.

5. Foundational Notes

What is an API?

API stands for:

Application Programming Interface

For now, think of an API as a way for one program to **ask another program for data**.

For example, imagine a website stores information about users.

Instead of opening the website manually, your Python program might ask:

```
Please give me information about user number 1.
```

The API could return:

```
{
  "id": 1,
  "name": "Asha",
  "city": "Bengaluru"
}
```

Your Python program can then use that information.

What is a URL?

A **URL** tells your program where the API is located.

Example concept:

```
https://example.com/users/1
```

Your Python program sends a request to that address.

Think of a URL like an **address on the internet**.

What is a request?

A **request** is a message sent by your Python program to a server.

For example:

```
Give me user number 1.
```

One common type of request is:

GET

A **GET request** usually means:

| "Please send me some information."

What is a response?

After receiving your request, the server sends something back.

That is called the **response**.

The response may contain:

- data
- a status code
- JSON
- error information

Example:

Request:
GET user 1

Response:
{
 "id": 1,
 "name": "Asha"
}

What is a status code?

A status code tells you whether the request worked.

One of the most important status codes is:

200

200 usually means:

| The request succeeded.

You may also see codes such as:

404

which usually means:

| The requested resource was not found.

Or:

500

which indicates that something went wrong on the server.

For today's lesson, the main idea is simply:

200 → success
anything else → handle carefully

What is a JSON response?

Many APIs return information like this:

```
{
  "id": 1,
  "name": "Asha",
  "city": "Bengaluru"
}
```

This is **JSON**.

The requests library can convert JSON response data into a Python structure that behaves much like the dictionaries you learned earlier.

Conceptually:

```
API JSON
  ↓
response.json()
  ↓
Python dictionary-like data
```

6. Client and Server — Very Simple Explanation

There are two useful words to understand.

Client

The **client** asks for something.

In today's example, your Python program is the client.

```
Python program → asks for data
```

Server

The **server** receives the request and sends a response.

```
Server → sends data back
```

Think about a restaurant.

You Client		Waiter/Kitchen Server
"I want food"	→	Request
Food	←	Response

For an API:

Python program		API server
GET request	→	receives request
JSON response	←	sends information

7. The requests Library

Python developers commonly use the **requests** package for simple web requests.

It is an external package, so you may need to install it first.

```
pip install requests
```

Then it can be imported into Python:

```
import requests
```

One important function is:

```
requests.get(...)
```

It sends a **GET request**.

For example, conceptually:

```
response = requests.get(api_url)
```

Notice that this only demonstrates the request step. It is **not the complete solution** for today's exercise.

8. Easy Example — Public/Sample API Concept

A beginner-friendly API might provide sample user information.

Imagine requesting:

```
https://example-api.com/users/1
```

The server could respond with:

```
{
  "id": 1,
  "name": "Neha",
  "city": "Mumbai"
}
```

Your Python program would follow this flow:

```
Create API URL
    ↓
Send GET request
    ↓
Check status code
    ↓
Convert response to JSON
    ↓
Access "name"
    ↓
Print Neha
```

A real learning API often behaves in exactly this general way.

9. Problem Statement

Your Day 68 Challenge

Create a small Python program that:

1. Stores the URL of a simple public/sample API.
2. Sends **one GET request** to the API.
3. Checks whether the request succeeded.
4. Converts the response into JSON data.
5. Reads **one value** from the JSON.
6. Prints that value.

For example, if the API returns:

```
{
  "id": 1,
  "name": "Asha",
  "city": "Bengaluru"
}
```

your program might print:

```
Asha
```

Keep the program small.

Do not add:

- authentication
- API keys
- pagination
- multiple requests
- async programming

10. Concepts Used

This exercise combines concepts you already know with a few new ones.

Concept	Purpose
Variable	Store the API URL
<code>requests</code>	Communicate with the API
<code>requests.get()</code>	Send a GET request
Response object	Store what the server sends back
Status code	Check whether the request worked
<code>if</code> statement	Handle success or failure
<code>.json()</code>	Convert JSON response into Python data
Dictionary key	Access one value
<code>print()</code>	Display the result

This is also why learning dictionaries before APIs was useful.

11. Thought Process

When solving an API problem, do not immediately think about all the code.

Think about the conversation between your program and the server.

Step 1: Where is the data?

You need an API URL.

```
API URL
```

Step 2: Ask for the data

Send a GET request.

```
Python → API
```

Step 3: Did the request work?

Check the response's status code.

```
status == 200 ?
```

Step 4: Convert the JSON

If the request succeeded:

```
JSON response
  ↓
Python data
```

Step 5: Find the value you need

For example:

```
"name"
```

Step 6: Print it

```
Asha
```

So your mental model should be:

```
URL
  ↓
GET request
  ↓
Response
  ↓
```

```
Check success
↓
JSON
↓
Dictionary key
↓
Print value
```

12. Beginner-Friendly Pseudocode

Do not worry about exact Python syntax yet.

```
START

import the requests library

store the API URL

send a GET request
store the response

IF the response status means success

    convert the response to JSON

    IF the required key exists

        get the value
        print the value

    ELSE

        print that the value was not found

ELSE

    print that the request failed

END
```

This pseudocode contains the entire **logic**, but you still need to translate it into Python yourself.

13. Suggested Solving Approach

Use a **simple request-response flow**.

Keep everything in this order:

1. Import requests
2. Store URL
3. Send GET request
4. Check status
5. Convert to JSON
6. Access one key
7. Print value

Avoid trying to process the JSON before checking whether the request succeeded.

A good beginner mindset is:

```
Request first  
Validate second  
Use data third
```

14. Easy Edge Cases

Edge Case 1: Internet Unavailable

Imagine your computer cannot connect to the internet.

Your program may not receive a normal API response.

Conceptually:

```
Python  
  ↓  
Internet unavailable  
  ↓  
Request cannot reach server
```

Later, you will learn proper exception handling for network problems.

For today's exercise, simply remember:

| A request is not guaranteed to succeed.

Edge Case 2: Unsuccessful Status Code

Suppose you receive:

```
404
```

You should not immediately try to use the expected JSON data.

Instead:

```
IF status is successful
    process JSON
ELSE
    print an error message
```

Possible output:

```
Request failed
```

Edge Case 3: Missing JSON Key

Suppose you expect:

```
{
  "name": "Asha"
}
```

but receive:

```
{
  "id": 1
}
```

Trying to access "name" directly may cause a problem.

You already learned a useful dictionary technique:

```
data.get("name")
```

You can use the same idea with JSON data after it has been converted into a Python dictionary.

15. Expected Flow and Output

Imagine the API sends:

```
{
  "id": 1,
  "name": "Asha",
  "city": "Bengaluru"
}
```

Your program should behave roughly like this:

```
Send GET request
  ↓
Receive response
  ↓
Check status code
  ↓
200
  ↓
Convert JSON
  ↓
Get "name"
  ↓
Print value
```

Expected output:

```
Asha
```

If the request fails, an acceptable output could be:

```
Request failed
```

If the key is missing:

```
Name not found
```

Hint Only

Start with these pieces and connect them yourself:

```
import requests

url = "...

response = requests.get(url)
```

Then investigate these two things:

```
response.status_code
```

and:

```
response.json()
```

Remember yesterday's dictionary lesson. Once JSON has been converted into Python data, ask yourself:

How would I get the value belonging to the "name" key from a dictionary?

That is the key idea behind today's challenge.

Day 68 takeaway

```
Python client
  ↓
GET request
  ↓
API server
  ↓
JSON response
  ↓
Python dictionary-like data
  ↓
Use the value
```

You now have the basic foundation for **collecting data from web APIs with Python** without yet needing authentication, pagination, or advanced networking.